

```

23     interaction_count INTEGER DEFAULT 0,
24     meetings_had INTEGER DEFAULT 0,
25
26     -- Social
27     linkedin_url VARCHAR(500),
28     twitter_handle VARCHAR(100),
29
30     -- Source
31     source VARCHAR(100),
32     source_detail TEXT,
33     added_at TIMESTAMPTZ DEFAULT NOW(),
34     updated_at TIMESTAMPTZ DEFAULT NOW(),
35
36     -- AI-Generated Insights (JSONB - flexible schema)
37     ai_insights JSONB DEFAULT '{
38         "suspected_pain_points": [],
39         "suspected_goals": [],
40         "buying_signals": [],
41         "decision_style": {},
42         "communication_style": {},
43         "anticipated_objections": [],
44         "competitive_intelligence": {},
45         "personality_traits": {},
46         "career_context": {},
47         "influence_analysis": {}
48     }'::jsonb,
49
50     -- Confirmed Facts (JSONB - ground truth)
51     confirmed_facts JSONB DEFAULT '{
52         "pain_points": [],
53         "goals": [],
54         "requirements": {},
55         "budget_timeline": {},
56         "decision_process": {},
57         "objections_raised": [],
58         "preferences": {},
59         "personal_context": {},
60         "competitors": {},
61         "success_metrics": []
62     }'::jsonb,
63
64     -- Insight Validation Tracking
65     insight_validation JSONB DEFAULT '{
66         "confirmed_insights": [],
67         "rejected_insights": [],
68         "pending_validation": []
69     }'::jsonb,
70
71     -- Custom Fields (user-defined, flexible)
72     custom_fields JSONB DEFAULT '{}'::jsonb,
73
74     -- Indexes
75     CONSTRAINT valid_relationship_strength CHECK
76     (relationship_strength >= 0 AND relationship_strength <= 100),
77     CONSTRAINT valid_responsiveness CHECK (responsiveness >= 0 AND
78     responsiveness <= 100)
79 );
80
81 -- Indexes for performance
82 CREATE INDEX idx_contacts_email ON contacts(email);
83 CREATE INDEX idx_contacts_company ON contacts(company);

```

```

82 CREATE INDEX idx_contacts_lifecycle_stage ON
contacts(lifecycle_stage);
83 CREATE INDEX idx_contacts_last_interaction ON
contacts(last_interaction);
84 CREATE INDEX idx_contacts_ai_insights ON contacts USING
GIN(ai_insights);
85 CREATE INDEX idx_contacts_confirmed_facts ON contacts USING
GIN(confirmed_facts);
86
87
88 -- =====
89 -- SALES REPS TABLE
90 -- =====
91 CREATE TABLE sales_reps (
92     rep_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
93     name VARCHAR(200) NOT NULL,
94     email VARCHAR(255) UNIQUE NOT NULL,
95     role VARCHAR(100),
96
97     -- Social Presence
98     linkedin_url VARCHAR(500),
99     linkedin_followers INTEGER,
100    twitter_handle VARCHAR(100),
101    twitter_followers INTEGER,
102
103    -- Personal Brand Metrics
104    content_published INTEGER DEFAULT 0,
105    engagement_rate DECIMAL(5,4),
106    thought_leader_score INTEGER, -- 0-100
107
108    -- Network Stats
109    direct_connections INTEGER,
110    second_degree_connections INTEGER,
111    connector_score INTEGER, -- 0-100
112
113    -- Goals & Performance
114    quota_monthly DECIMAL(12,2),
115
116    -- Preferences & Style (JSONB)
117    preferences JSONB DEFAULT '{
118        "communication_style": "consultative",
119        "preferred_channels": ["email", "linkedin"],
120        "working_hours": {
121            "timezone": "America/Los_Angeles",
122            "available": ["9:00-17:00"]
123        },
124        "signature": "",
125        "email_templates": {}
126    }'::jsonb,
127
128    -- Performance Metrics (JSONB)
129    performance JSONB DEFAULT '{
130        "win_rate": 0,
131        "avg_deal_size": 0,
132        "avg_sales_cycle_days": 0,
133        "reply_rate": 0,
134        "meeting_book_rate": 0
135    }'::jsonb,
136
137    created_at TIMESTAMPTZ DEFAULT NOW(),
138    updated_at TIMESTAMPTZ DEFAULT NOW(),
139

```

```

140     CONSTRAINT valid_thought_leader_score CHECK
(thought_leader_score >= 0 AND thought_leader_score <= 100),
141     CONSTRAINT valid_connector_score CHECK (connector_score >= 0 AND
connector_score <= 100)
142 );
143
144
145 -- =====
146 -- SEGMENTATIONS TABLE (NEW)
147 -- =====
148 CREATE TABLE segmentations (
149     segmentation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
150     name VARCHAR(200) NOT NULL,
151     description TEXT,
152
153     -- Segment Priority (1-10)
154     priority INTEGER NOT NULL DEFAULT 5,
155
156     -- Segmentation Rules (JSONB - flexible criteria)
157     criteria JSONB NOT NULL DEFAULT '{
158         "filters": [],
159         "scoring_rules": [],
160         "exclusions": []
161     }'::jsonb,
162
163     -- ICP Definition for this segment
164     icp_definition JSONB DEFAULT '{
165         "ideal_title_patterns": [],
166         "ideal_company_size": {},
167         "ideal_industries": [],
168         "ideal_tech_stack": [],
169         "key_attributes": {}
170     }'::jsonb,
171
172     -- Playbook reference
173     playbook_id UUID REFERENCES playbooks(playbook_id),
174
175     -- Ownership
176     created_by UUID REFERENCES sales_reps(rep_id),
177     team_id UUID, -- Optional: for team-level segments
178
179     -- Status
180     is_active BOOLEAN DEFAULT true,
181
182     -- Metadata
183     created_at TIMESTAMPTZ DEFAULT NOW(),
184     updated_at TIMESTAMPTZ DEFAULT NOW(),
185
186     CONSTRAINT valid_priority CHECK (priority >= 1 AND priority <=
10)
187 );
188
189 CREATE INDEX idx_segmentations_priority ON segmentations(priority
DESC);
190 CREATE INDEX idx_segmentations_active ON segmentations(is_active);
191 CREATE INDEX idx_segmentations_criteria ON segmentations USING
GIN(criteria);
192
193
194 -- =====
195 -- CONTACT_SEGMENT_SCORES TABLE
196 -- =====

```

```

197 -- Many-to-many with scoring
198 CREATE TABLE contact_segment_scores (
199     id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
200     contact_id UUID REFERENCES contacts(contact_id) ON DELETE
CASCADE,
201     segmentation_id UUID REFERENCES segmentations(segmentation_id) ON
DELETE CASCADE,
202
203     -- Fit Score (0-10)
204     fit_score DECIMAL(3,1) NOT NULL,
205
206     -- Score Breakdown (JSONB)
207     score_breakdown JSONB DEFAULT '{
208         "title_match": 0,
209         "company_size_match": 0,
210         "industry_match": 0,
211         "tech_stack_match": 0,
212         "engagement_level": 0,
213         "buying_signals": 0
214     }'::jsonb,
215
216     -- Auto-calculated or manual
217     score_type VARCHAR(20) DEFAULT 'auto', -- auto | manual
218
219     -- Status in this segment
220     status VARCHAR(50) DEFAULT 'qualified', -- qualified | active |
paused | excluded
221
222     -- Campaign enrollment for this segment
223     enrolled_in_campaign BOOLEAN DEFAULT false,
224     current_campaign_id UUID,
225
226     -- Timestamps
227     scored_at TIMESTAMPTZ DEFAULT NOW(),
228     last_updated TIMESTAMPTZ DEFAULT NOW(),
229
230     UNIQUE(contact_id, segmentation_id),
231     CONSTRAINT valid_fit_score CHECK (fit_score >= 0 AND fit_score <=
10)
232 );
233
234 CREATE INDEX idx_contact_segment_contact ON
contact_segment_scores(contact_id);
235 CREATE INDEX idx_contact_segment_segmentation ON
contact_segment_scores(segmentation_id);
236 CREATE INDEX idx_contact_segment_score ON
contact_segment_scores(fit_score DESC);
237 CREATE INDEX idx_contact_segment_status ON
contact_segment_scores(status);
238
239
240 -- =====
241 -- PLAYBOOKS TABLE (NEW)
242 -- =====
243 CREATE TABLE playbooks (
244     playbook_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
245     name VARCHAR(200) NOT NULL,
246     description TEXT,
247
248     -- Playbook Type
249     playbook_type VARCHAR(50) NOT NULL, -- nurture | outreach |
re_engagement | upsell

```

```

250
251 -- Playbook Configuration (JSONB)
252 config JSONB NOT NULL DEFAULT '{
253     "stages": [],
254     "messaging_strategy": {},
255     "timing_rules": {},
256     "channel_sequence": []
257 }'::jsonb,
258
259 -- Stage-specific templates
260 templates JSONB DEFAULT '{
261     "lead": {},
262     "opportunity": {},
263     "customer": {}
264 }'::jsonb,
265
266 -- Success Metrics
267 metrics JSONB DEFAULT '{
268     "target_reply_rate": 0,
269     "target_meeting_rate": 0,
270     "avg_time_to_response": 0
271 }'::jsonb,
272
273 -- Performance Tracking
274 performance JSONB DEFAULT '{
275     "contacts_enrolled": 0,
276     "total_sent": 0,
277     "total_replies": 0,
278     "total_meetings": 0,
279     "reply_rate": 0,
280     "meeting_rate": 0
281 }'::jsonb,
282
283 -- Ownership
284 created_by UUID REFERENCES sales_reps(rep_id),
285 is_template BOOLEAN DEFAULT false, -- System templates vs custom
286
287 -- Status
288 is_active BOOLEAN DEFAULT true,
289
290 created_at TIMESTAMPTZ DEFAULT NOW(),
291 updated_at TIMESTAMPTZ DEFAULT NOW()
292 );
293
294 CREATE INDEX idx_playbooks_type ON playbooks(playbook_type);
295 CREATE INDEX idx_playbooks_active ON playbooks(is_active);
296
297
298 -- =====
299 -- PLAYBOOK_STAGES TABLE
300 -- =====
301 CREATE TABLE playbook_stages (
302     stage_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
303     playbook_id UUID REFERENCES playbooks(playbook_id) ON DELETE
304     CASCADE,
305
306     -- Stage Info
307     stage_order INTEGER NOT NULL,
308     stage_name VARCHAR(100) NOT NULL,
309     stage_type VARCHAR(50) NOT NULL, -- email | linkedin | call |
    wait | conditional

```

```

310  -- Trigger Conditions
311  trigger_conditions JSONB DEFAULT '{
312      "wait_duration": null,
313      "wait_for_event": null,
314      "conditions": []
315  }'::jsonb,
316
317  -- Content Configuration
318  content_config JSONB DEFAULT '{
319      "template_id": null,
320      "personalization_rules": [],
321      "ai_generation_prompt": null,
322      "fallback_template": null
323  }'::jsonb,
324
325  -- Success Criteria (when to advance)
326  success_criteria JSONB DEFAULT '{
327      "on_reply": "advance",
328      "on_open": "continue",
329      "on_click": "continue",
330      "on_timeout": "next_stage"
331  }'::jsonb,
332
333  created_at TIMESTAMPTZ DEFAULT NOW(),
334
335  UNIQUE(playbook_id, stage_order)
336 );
337
338 CREATE INDEX idx_playbook_stages_playbook ON
playbook_stages(playbook_id);
339 CREATE INDEX idx_playbook_stages_order ON
playbook_stages(playbook_id, stage_order);
340
341
342  -- =====
343  -- INTERACTIONS LOG TABLE
344  -- =====
345  CREATE TABLE interactions (
346      interaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
347      contact_id UUID REFERENCES contacts(contact_id) ON DELETE
CASCADE,
348      sales_rep_id UUID REFERENCES sales_reps(rep_id),
349
350      -- Campaign Context
351      campaign_id UUID,
352      segmentation_id UUID REFERENCES segmentations(segmentation_id),
353      playbook_id UUID REFERENCES playbooks(playbook_id),
354      stage_id UUID REFERENCES playbook_stages(stage_id),
355
356      -- Interaction Type
357      interaction_type VARCHAR(50) NOT NULL,
358      -- email_sent | email_opened | email_clicked | email_replied |
359      -- linkedin_message | linkedin_accept | call_scheduled |
meeting_held |
360      -- content_download | website_visit
361
362      -- Content
363      content JSONB DEFAULT '{
364          "subject": null,
365          "body": null,
366          "sentiment": null,
367          "intent": null,

```

```
368     "key_topics": [],
369     "mentioned_pain_points": [],
370     "mentioned_goals": [],
371     "objections_raised": []
372 }'::jsonb,
373
374 -- Metadata
375 channel VARCHAR(50), -- email | linkedin | phone | in_person
376 direction VARCHAR(20), -- inbound | outbound
377
378 -- AI Analysis
379 ai_analysis JSONB DEFAULT '{
380     "intent_classification": null,
381     "sentiment_score": null,
382     "buying_signal_detected": false,
383     "next_best_action": null,
384     "confidence": null
385 }'::jsonb,
386
387 -- Impact
388 rank_change DECIMAL(3,1) DEFAULT 0,
389 segment_changes JSONB DEFAULT '[]'::jsonb,
390
391 -- Timestamps
392 occurred_at TIMESTAMPTZ DEFAULT NOW(),
393 processed_at TIMESTAMPTZ,
394
395 CONSTRAINT valid_rank_change CHECK (rank_change >= -10 AND
rank_change <= 10)
396 );
397
398 CREATE INDEX idx_interactions_contact ON interactions(contact_id);
399 CREATE INDEX idx_interactions_campaign ON
interactions(campaign_id);
400 CREATE INDEX idx_interactions_type ON
interactions(interaction_type);
401 CREATE INDEX idx_interactions_occurred ON interactions(occurred_at
DESC);
402 CREATE INDEX idx_interactions_content ON interactions USING
GIN(content);
```

Implementation Plan

CosmoAgents Implementation Specification

Claude Agent SDK Architecture with Shared Skills

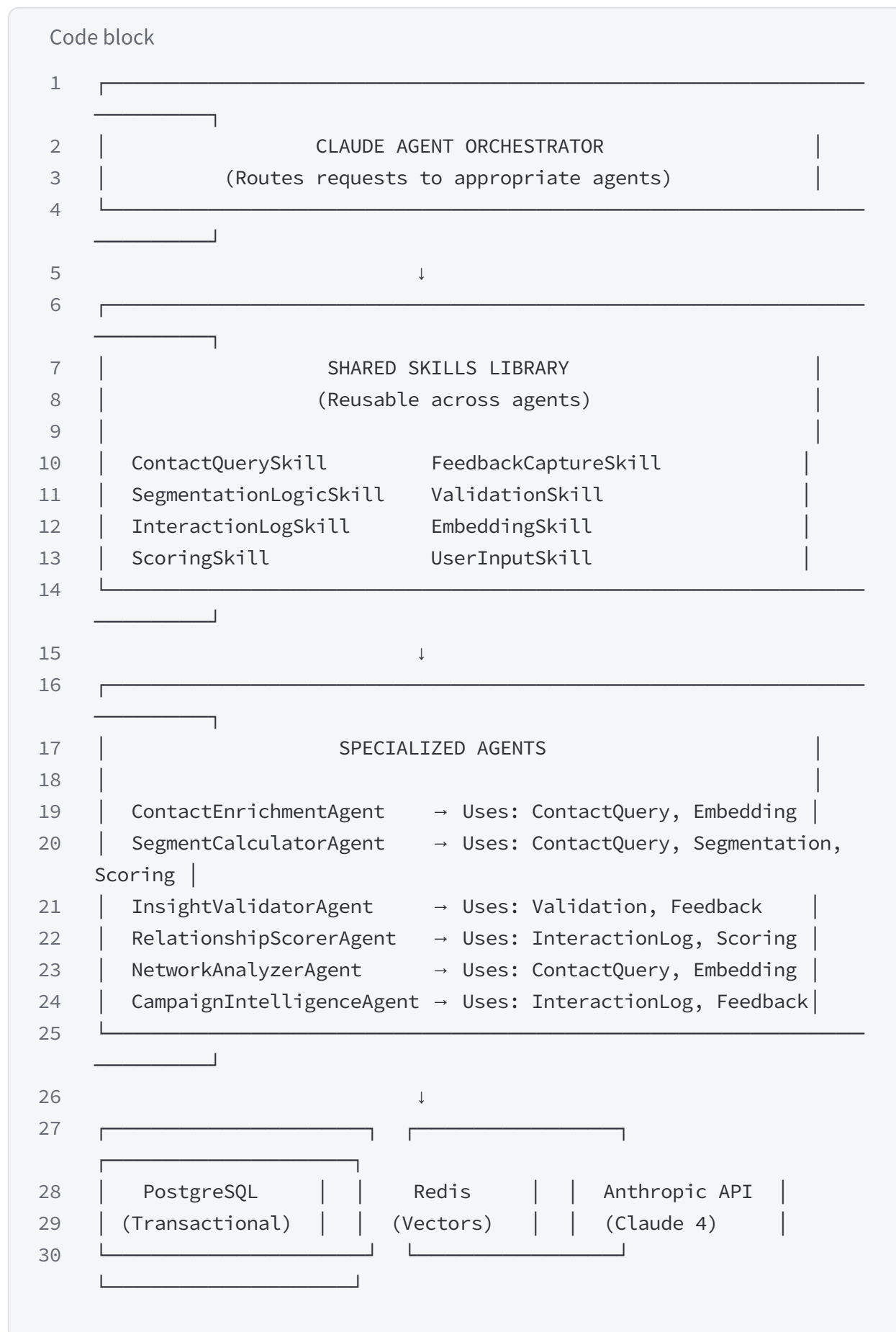
Framework: Claude Agent SDK
Documentation: <https://platform.claude.com/docs/en/agent-sdk/overview>
Architecture: Skill-Based Agents + PostgreSQL + Redis

TABLE OF CONTENTS

- 1. Architecture Overview
- 2. Shared Skills Library
- 3. Agent Specifications

2.4 ARCHITECTURE OVERVIEW

1.1 Claude Agent SDK Structure



1.2 Skill-Based Architecture Benefits

Why Skills?

- **✓ Reusability:** ContactQuerySkill used by all 6 agents
- **✓ Maintainability:** Update filtering logic once, affects all agents
- **✓ Testability:** Test skills independently

-  **Composability:** Agents combine skills to solve complex tasks

Example Agent Composition:

Code block

```
1  from agent_sdk import Agent
2
3  class SegmentCalculatorAgent(Agent):
4      """
5      Calculates contact fit scores for segments
6      """
7
8      skills = [
9          ContactQuerySkill(),      # Fetch contact data
10         SegmentationLogicSkill(), # Apply criteria
11         ScoringSkill(),           # Calculate scores
12         FeedbackCaptureSkill()    # Learn from user input
13     ]
14
15     async def run(self, contact_id: str, segmentation_id: str):
16         # Use skills to accomplish task
17         contact = await self.use_skill("contact_query",
18 contact_id=contact_id)
19         segment = await self.use_skill("contact_query",
20 segmentation_id=segmentation_id)
21
22         # Apply user-defined logic stored in segment criteria
23         score = await self.use_skill("segmentation_logic",
24 contact=contact,
25 criteria=segment.criteria)
26
27         # Store result
28         await self.use_skill("scoring", save_score=score)
29
30         return score
```

2.2 SHARED SKILLS LIBRARY

2.1 Skill: ContactQuerySkill

Purpose: Query and filter contacts with user-defined criteria

File: skills/contact_query_skill.py

Code block

```
1  from agent_sdk import Skill, tool
2  from typing import Dict, List, Optional, Any
3  import asyncpg
4
5  class ContactQuerySkill(Skill):
6      """
7      Unified contact querying with PostgreSQL JSONB queries
8
9      Supports:
10     - Get contact by ID
11     - Filter contacts with complex criteria
12     - Semantic search (Redis vectors)
13     - Custom user-defined filters
```

```

14     """
15
16     name = "contact_query"
17     description = "Query contacts with flexible filters"
18
19     def __init__(self, db_pool: asyncpg.Pool, redis_client):
20         self.db = db_pool
21         self.redis = redis_client
22
23     @tool
24     async def get_contact(self, contact_id: str) -> Dict:
25         """
26         Fetch single contact with full profile
27
28         Args:
29             contact_id: UUID of contact
30
31         Returns:
32             Contact dict with profile JSONB data
33         """
34         query = """
35             SELECT
36                 id, user_id, email, first_name, last_name,
37                 company, job_title, phone,
38                 profile, tags, do_not_contact,
39                 created_at, updated_at
40             FROM contacts
41             WHERE id = $1 AND is_deleted = false
42         """
43
44         row = await self.db.fetchrow(query, contact_id)
45
46         if not row:
47             raise ValueError(f"Contact {contact_id} not found")
48
49         return dict(row)
50
51     @tool
52     async def filter_contacts(
53         self,
54         user_id: str,
55         filters: Dict[str, Any],
56         sort_by: Optional[str] = None,
57         limit: int = 100
58     ) -> List[Dict]:
59         """
60         Filter contacts with dynamic JSONB queries
61
62         Args:
63             user_id: User ID (for multi-tenancy)
64             filters: Dynamic filter criteria
65             sort_by: Sort field (e.g., "relationship_strength")
66             limit: Max results
67
68         Filter examples:
69             {
70                 "lifecycle_stage": "Opportunity",
71                 "relationship_strength_score": {">=": 70},
72                 "segmentation.priority_score": {">": 80},
73                 "ai_insights.buying_signals": {"length": {">": 2}},
74                 "confirmed_facts.budget_timeline.budget_available":
{">=": 50000},

```